Explicit Discretization: How Transformers Beat XGBoost on Calibrated Tabular Forecasting

Yael S. Elmatad
yael.elmatad@gmail.com

Code: `https://github.com/yaelelmatad/RunTime-Public`

Abstract

Gradient boosting still dominates Transformers [1] on tabular benchmarks. Our tokenizer uses a deliberately simplistic discretized vocabulary so we can highlight how even basic tokenization unlocks the power of attention on tabular features, yet it already outperforms tuned gradient boosting when combined with Gaussian smoothing. Our solution discretizes environmental context while smoothing labels with adaptive Gaussians, yielding calibrated PDFs. On 600K entities (5M training examples) we outperform tuned XGBoost [2] by 10.8% (35.94s vs 40.31s median MAE) and achieve KS=0.0045 with the adaptive-σ checkpoint selected to minimize KS rather than median MAE. Ablations confirm architecture matters: losing sequential ordering costs about 2.0%, dropping the time-delta tokens costs about 1.8%, and a stratified calibration analysis reveals where miscalibration persists.

1 Introduction

XGBoost [2] remains the default choice for tabular data because its axis-aligned splits create discrete regimes that Transformers usually miss. RunTime treats each career as a causal stride of discretized tokens, respecting irregular time deltas so the model learns a full Probability Density Function (PDF) rather than a point estimate. Our approach tackles this by discretizing both inputs and outputs, training with adaptive Gaussian-soft targets, and explicitly representing cadence with time delta tokens. Our contributions are:

1. Architectural insight that discrete regimes, not bigger Transformers, unlock tabular performance;
2. Gaussian smoothing - both fixed (σ) and adaptive ($\sigma_i = \sqrt{\sigma_{\text{floor}}^2 + (k \cdot w_i)^2}$, with $w_i = b_i^{\text{end}} - b_i^{\text{start}}$) that scales the smoothing strength with bin width while enforcing a minimum floor;
3. Empirical win: 10.8% lower median MAE than tuned XGBoost plus KS=0.0045 calibration (KS optimized via the adaptive-σ best-KS checkpoint);
4. Analysis methodology including stratified calibration to diagnose residual miscalibration; and
5. Sequence-aware modeling that explicitly represents cadence with time-delta tokens and relies on entity-disjoint splits to keep temporal dependencies honest for unseen runners.

These design decisions build on prior work demonstrating the effectiveness of discretization for forecasting [3, 4, 5], extending the discretized training philosophy from regular time series to irregular, tabular trajectories with entity-level splits and adaptive Gaussian smoothing.

2 Related Work

2.1 Tabular Transformers

Despite Transformers' flexibility, XGBoost remains dominant on tabular benchmarks. Gorishniy et al. [6] show the FT-Transformer loses to XGBoost on 11 datasets, and the TabTransformer [7] and TabNet [8] improve over prior neural methods but stay generally competitive with rather than superior to tree-based models. These models suffer because trees naturally produce piece-wise constant (irregular) decision boundaries through axis-aligned splits, whereas neural networks are inherently smooth function approximators that struggle with the irregular patterns common in tabular data [9, 10]. Our work addresses the same failure mode by making discretization explicit and combining it with distributional training.

2.2 Ordinal Regression and Calibration

Ordinal regression benefits from soft targets that preserve ordering. Diaz & Marathe [11] introduce soft ordinal labels represented as probability vectors over ordinal categories. Calibration literature (Guo et al. [12]) demonstrates the need for stratified diagnostics beyond global metrics, showing that miscalibration can vary across confidence levels. Kuleshov et al. [13] develop calibrated regression methods that produce well-calibrated predictive distributions. Our approach integrates Gaussian-integrated targets and stratified calibration during training, rather than relying on post-hoc temperature scaling.

2.3 Event Trajectories and Temporal Point Processes

Modeling irregular event trajectories draws on temporal point process foundations (Du et al. [14], Shchur et al. [15], Zuo et al. [16]), which highlight the need to respect both the event content and the timing gaps. Chronos introduced a similar discretization thesis for regular time series, showing that Transformers trained with discretized horizons match or exceed state-of-the-art forecasting models when each time slot is binned (Ansari et al. [4]). Chronos-2 extends that idea toward universal forecasting by mixing discrete value heads with continuous time embeddings (Ansari et al. [5]). Our method extends these insights to heterogeneous bins, irregular histories (requiring explicit time-delta tokens), and entity-disjoint evaluation (preventing individual memorization), bringing the discretization thesis from forecasting into tabular settings rather than strictly generative time-series modeling.

2.4 Discretization for Forecasting

Rabanser et al. [3] empirically demonstrate that treating forecasting as classification over discretized bins almost always outperforms regression, attributing the gains to the implicit regularization of binning and the flexibility of distributional outputs. Their method uses hard one-hot targets and assumes uniform bin widths, which ignores ordinal structure and limits handling for wide bins. We extend this line of work by preserving ordinal information through Gaussian-integrated soft targets and by introducing non-adaptive (fixed σ) and adaptive smoothing ($\sigma_i = \sqrt{\sigma_{\text{floor}}^2 + (k \cdot w_i)^2}$) to adapt to the heterogeneous target bin widths that arise in binned tabular data.

3 Method

3.1 Problem Setup

Each training example is a sequence of past races for a single runner. Each race contributes:

- environmental conditions (temperature, wind speed, feels-like temperature, humidity, and qualitative conditions such as rain or snow),
- race metadata (distance),
- demographics (age, gender), and
- temporal gaps (`weeks_since_last`, `weeks_to_target`) plus the observed pace.

The dataset benefits from the NYRR 9+1 Program qualification history [17], which ensures a consistent set of marathon and distance event completions per runner, and environmental conditions are sourced from the Visual Crossing Weather API [18]. We discretize pace into 270+ bins and frame the task as predicting the soft distribution across those bins.

Entity-disjoint evaluation. We split 600K runners into train/validation/test (270K/30K/60K runners, 2.25M/250K/500K examples, 5M total available) with no runner overlap. This prevents the model from memorizing individuals and puts the focus on generalization to unseen runners.

3.2 Discretization Strategy

Environmental inputs (temperature, humidity, wind) and the modeled pace are binned via a balanced (quantile-based) quantization procedure so each bin contains roughly the same number of examples, mirroring how tree splits carve regime-specific decision regions. Bins are capped in width, and overly wide extremes are recursively split until their widths stay somewhat comparable to the rest of the vocabulary, keeping the softmax landscape expressive even for rare outcomes. We treat race distance and the two time deltas (`weeks_since_last`, `weeks_to_target`) as tokens directly; they are not quantized via the balanced procedure but are instead encoded from the raw values so the model sees the real timing information. This gives us two token flavors: **quantized continuous** tokens (pace, weather, etc.) representing discretized numeric ranges, and **categorical** tokens (gender, weather descriptor, etc.) that encode semantics. Everything in the model—including environmental states, pace bins, demographic cues, and temporal gaps—is represented as a language token, and each event block concatenates the appropriate mix of tokens followed by cadence: `[features+demographics][pace][d_next][d_fin]`. The 327-token window supports up to 30 events and ends with the target pace token to avoid leakage.

3.3 Gaussian-integrated Soft Targets

Following Rabanser et al. [3], we treat the prediction as a classification over discretized bins but replace their hard one-hot targets with Gaussian-integrated soft targets that preserve ordinality:

$$T_i = \int_{b_i^{\text{start}}}^{b_i^{\text{end}}} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - y_{\text{true}})^2}{2\sigma^2}\right) dx,$$

so bins near the true pace receive consistent credit, preserving ordinal structure.

3.4 Adaptive Smoothing

Fixed σ values work well for narrow bins (many middle bins are only 1–3 seconds wide): a 3-second Gaussian produces soft targets for bins of comparable width yet behaves almost one-hot once bin widths significantly exceed the value of σ. We therefore scale smoothing via the production quadrature-inspired rule:

$$\sigma_i = \sqrt{\sigma_{\text{floor}}^2 + (k \cdot w_i)^2},$$

where $w_i = b_i^{\text{end}} - b_i^{\text{start}}$ is the width of bin i which is the bin in which the target value lands, σ_{floor} enforces a minimum smoothing width, and k controls how aggressively the bin width influences the smoothing. This keeps narrow bins sharp while allowing broad bins to receive proportionally more mass, and the parameter pair $(\sigma_{\text{floor}}, k)$ is tuned on validation data and cached per bin via the training config. The results reported in this paper use $\sigma_{\text{floor}} = 2.7$ and $k = 1.5$.

3.5 Architecture

A causal Transformer [1] (6 layers, 8 heads, 512-dimensional embeddings) processes the token stream. Pace tokens sit before the cadence tokens, and we mask attention to enforce causality. The decoder-style transformer ingests the fixed 11-token stride grammar so every event block is handled like auto-regressive language modeling, producing logits over pace bins that are trained with the Gaussian-smoothed cross-entropy objective described earlier (standard cross-entropy only appears in a specific ablation).

Each event block follows a strict grammar: First the environmental, race specific, and demographic tokens (temperature, humidity, wind, feels-like temperature, rain/clear/other qualitative conditions, gender, age, discretized race distance) then the modeled outcome—pace—sits immediately after the contextual features so the model can learn how environmental/demographic states map into performance, and only then do the two cadence tokens (`weeks_since_last`, `weeks_to_target`) appear to signal the temporal gaps. A block therefore reads `[race features and demographics][pace][d_next][d_fin]`, and at inference time the model autoregressively predicts the final pace token given the preceding context in each stride. Padding tokens fill unused slots so every block keeps the same stride length, which keeps positional encodings (sinusoidal) meaningful even when histories vary from one to thirty events.

4 Experiments

4.1 Setup

Entity-disjoint splits ensure zero leakage. The test set comprises 60K runners (500K predictions). Evaluation includes MAE, RMSE, and the Kolmogorov-Smirnov statistic for calibration.

4.2 Results

Table 1 summarizes the median MAE gap between the full model and the baselines.

4.3 Training Configuration

Table 2 collects the core training settings so the reported convergence behavior can be reproduced at scale.

Table 1: Main results

Model	Mean MAE	Median MAE	Mode MAE	Median RMSE
Full model ($\sigma = 3$)	**36.54**	**35.94**	**38.50**	**71.83**
XGBoost (tuned)	40.31	40.31	40.31	73.15
Naive mean	52.72	52.72	52.72	88.16
Riegel formula	49.74	49.74	49.74	94.71

Table 2: Training configuration for the fixed $\sigma = 3$ sweep. RunTime is a decoder-style causal Transformer.

Parameter	Value
Transformer width	512
Inner feed forward size	2,048
Dropout after attention/FFN	0.12
Max races to consider	30
Attention heads	8
Decoder-only Transformer layers	6
Per-GPU batch size	64
AdamW base learning rate	1e-4
Gaussian smoothing σ seconds	3
AdamW weight decay	0.00118

Ablation configurations These MAE statistics come from the dedicated ablation sweep with reduced batch sizes, so they represent slightly more converged checkpoints than the main training runs. We report mean/median/mode to compare the distributional forecasts against tuned XGBoost (median MAE from the sweep) as well as the Naive mean (52.72s) and Riegel (49.74s) baselines, keeping every summary in the same natural units as the ablation runs. The optimizer is AdamW [19], which lets us independently tune weight decay alongside the learning rate for better generalization.

Riegel formula baseline The Riegel formula extrapolates a runner's performance across distances by assuming a power-law relation between finishing time and distance [20]. We take each runner's penultimate race as a reference point, normalize its performance by distance, and roll that history forward through the formula to predict the current target pace; this provides a lightweight, physics-inspired benchmark that captures pacing consistency without learning attention weights.

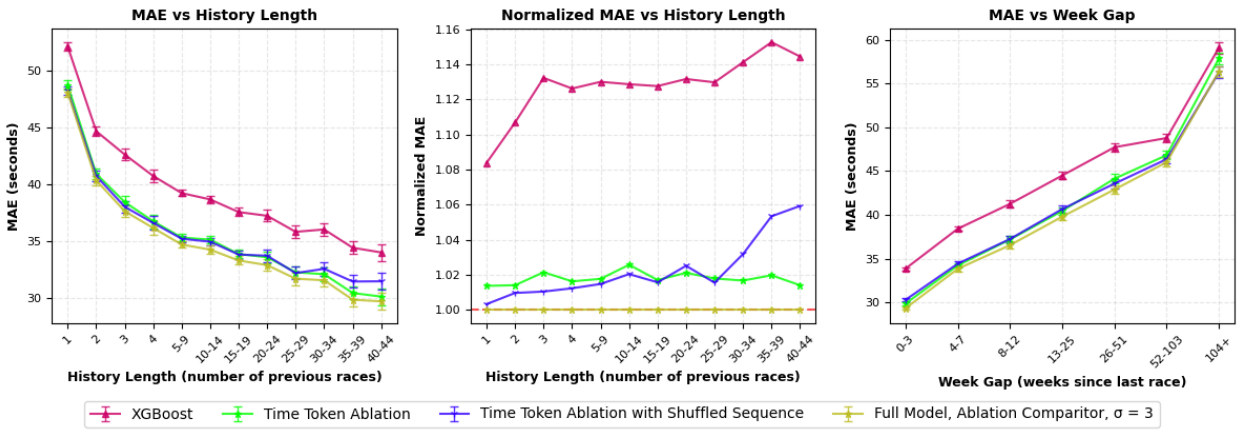

Figure 1: Top: MAE vs. history length. Middle: normalized MAE showing the ablation gap relative to the full model. Bottom: MAE vs. week gap between the penultimate and target race, highlighting temporal staleness.

4.4 Ablations

Table 3: Ablation MAE statistics ($\sigma = 3$) and wall-clock costs to best MAE.

Configuration	Mean MAE	Median MAE	Mode MAE	Wall clock (h)
Time token ablation	37.24	36.58	39.28	107
Time token ablation, shuffled sequence	37.23	36.65	39.73	145
Full model ($\sigma = 3$)	**36.54**	**35.94**	**38.50**	**60**

Table 3 validates that architectural choices—not just model capacity—drive the gains. Figure 1 (middle) shows performance relative to the full model, and the paragraphs below break the contributions down.

Time delta tokenization (≈1.8% gain). Removing explicit time delta tokens raises median MAE from 35.94s to 36.58s (≈1.8%) while the wall-clock to converge jumps from 60h to 107h. This shows that temporal features simultaneously sharpen accuracy and accelerate training.

Even though the accuracy hit remains modest in aggregate, the bottom panel of Figure 1 shows the degradation grows with longer histories, highlighting the cadence tokens' role in extrapolating far-out predictions. At the same time the full model converges much faster (60h vs 107h) perhaps because the explicit deltas keep the sequence grounded.

Temporal ordering (≈2.0% gain). Shuffling the race history increases median MAE from 35.94s to 36.65s (≈2.0%), reinforcing that preserving chronological order lets the Transformer capture cadence-dependent progression patterns that random sequence destroys.

Temporal generalization. Figure 1 (bottom) plots MAE vs. the week gap between a runner's penultimate and target race. All variants worsen as the gap grows—closing the gap between the Transformers and XGBoost—suggesting a convergence toward baseline performance at long gaps. The middle panel also shows that the relative gap between the shuffled and full models widens as history length increases, confirming that preserving chronological order helps the Transformer learn progression patterns that generalize better than tree-based methods.

4.5 Calibration Analysis

Figure 2 shows the updated Q–Q plot staying near uniform (KS=0.0045 from the best-KS adaptive-σ checkpoint), Figure 3 highlights where the predicted probability concentrates its mass, and Figure 4 traces eight decile-specific curves that reveal residual drift for slower runners. These diagnostics demonstrate overall calibration while pointing to the performance-dependent deviations that adaptive smoothing targets, and

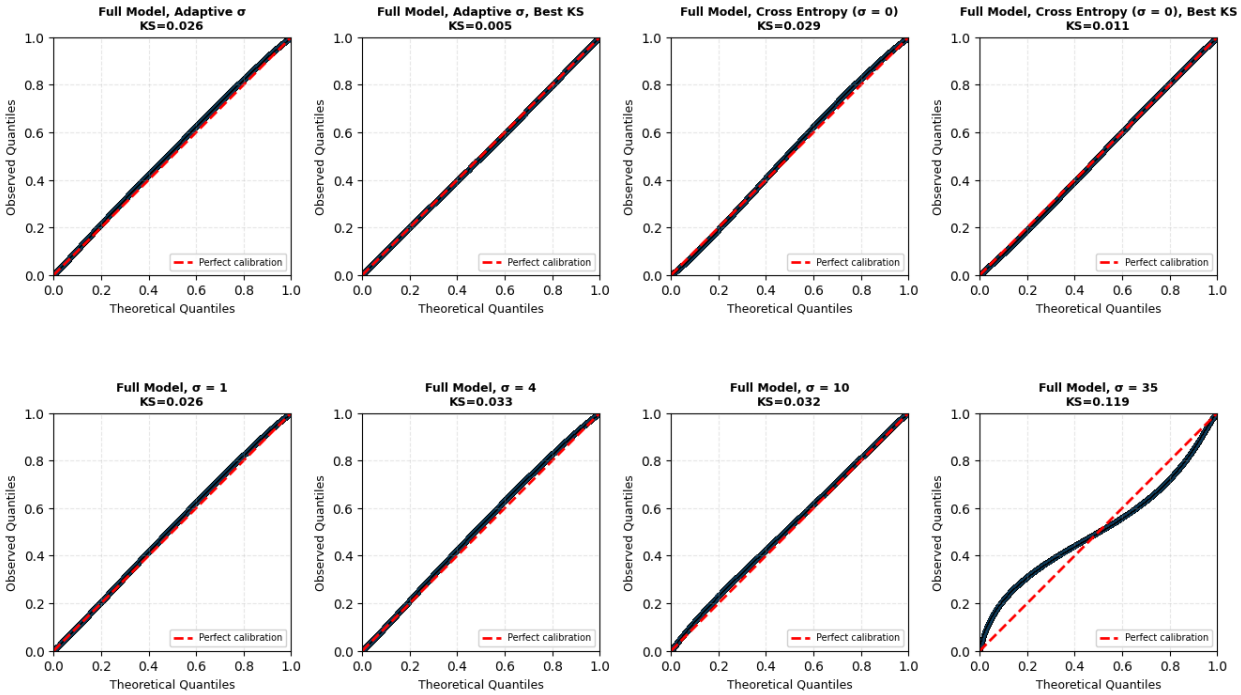

Figure 2: Q–Q plot showing distributions across various percentiles from $\sigma = 0$ (cross entropy) to $\sigma = 35$ (wide Gaussian).

all three plots were generated using roughly 250,000 evaluation points across 10 bins per decile to keep the comparisons consistent. Importantly this calibration arrives without any post-training temperature scaling—the adaptive σ schedule effectively plays the role of an in-training temperature, and its dependence on bin width and statistics could also motivate a post-training "adaptive temperature" refinement that explicitly compensates for the granular, semi-quantized vocabulary.

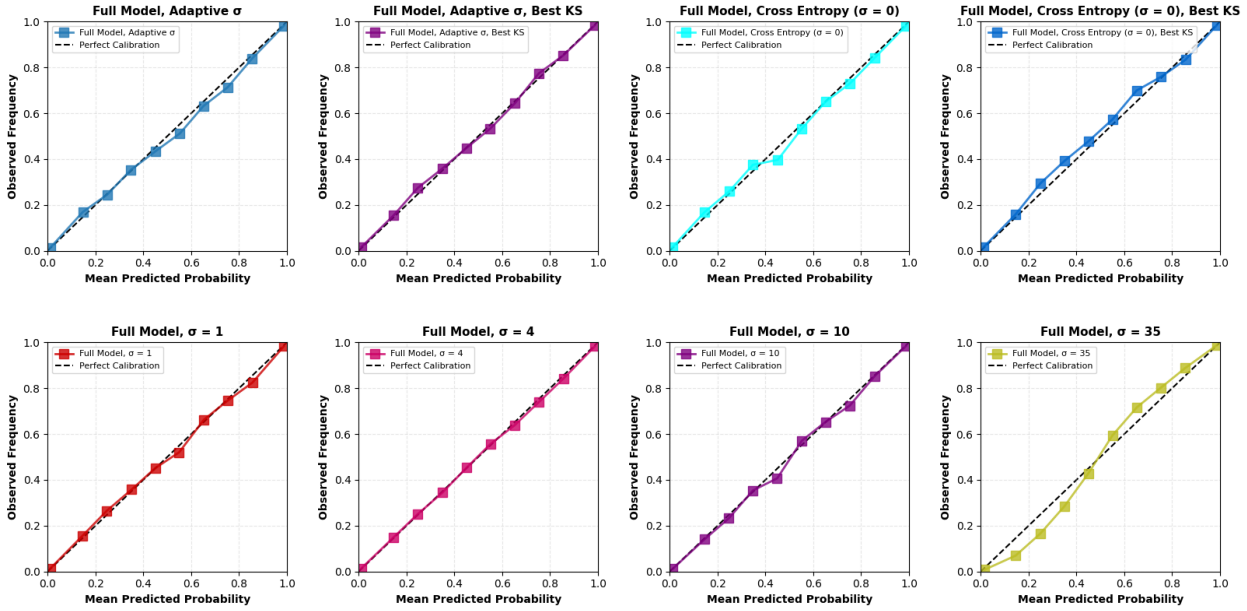

Figure 4: Calibration plots highlighting how different smoothing choices affect residual drift across percentiles.

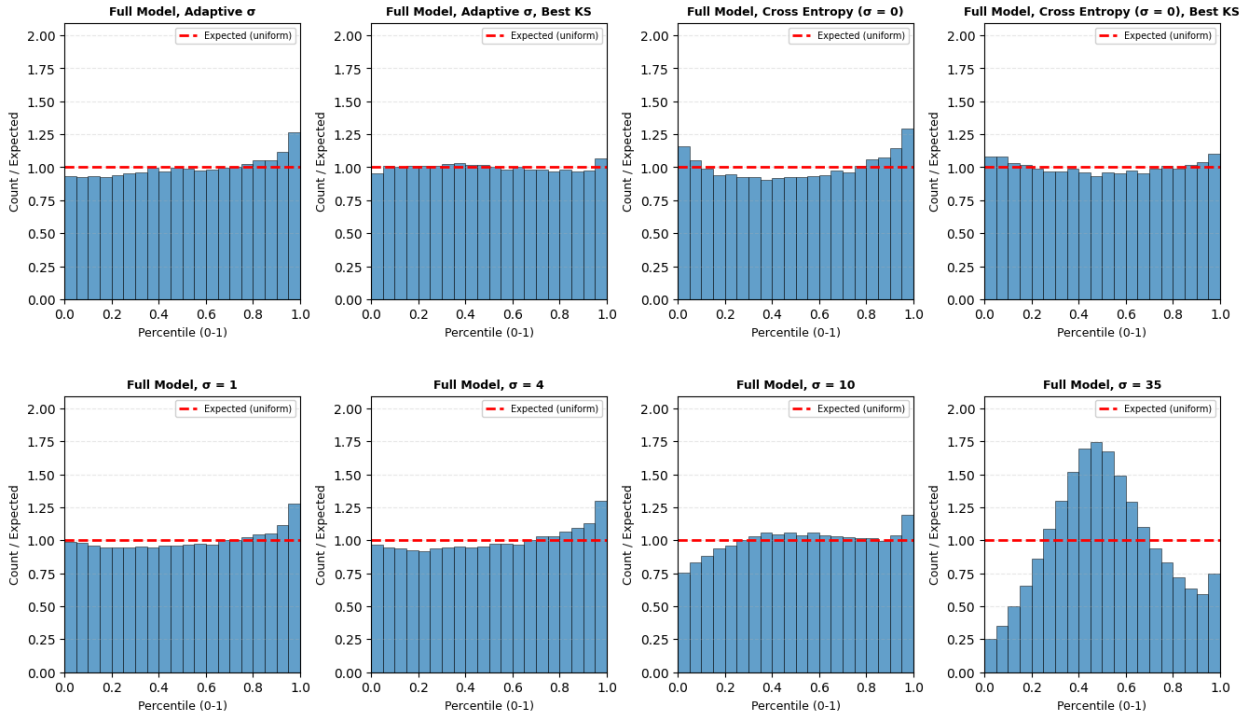

Figure 3: Quantile calibration emphasizing where predicted densities concentrate. Note that small σ values concentrate predictions at the extremes, suggesting model overconfidence, whereas large σ values appear underconfident, with larger probability mass in the middle.

Model leaderboard summary Table 4 reports the validation MAE/RMSE statistics collected for eight variants of the full model used in the calibration sweep. These runs were configured for fast iteration (larger

batch sizes, adjusted learning rates, shorter sweeps) and should not be compared against the more converged $\sigma = 3$ benchmark in Table 1; rather, they exist to compare relative behavior under different smoothing/KS choices. Because the median MAE in Table 1 comes from the extensively trained $\sigma = 3$ checkpoint (35.94s), the entries in this sweep deliberately report different medians, and the "best KS" checkpoint can therefore have a higher MAE even though its KS is lowest. Rows marked "Best KS" refer to the checkpoint selected for the smallest KS, i.e., the calibration-optimized partner for that loss variant rather than the run with the minimum median MAE. All other values of σ had optimized KS values between the adaptive σ sweep and the cross entropy ($\sigma = 0$) sweep without improvement in MAE.

Table 4: Calibration sweep leaderboard (seconds). Lower is better; values were collected on short, cost-aware sweeps and should only be compared internally.

	KS	MAE			RMSE		
Model	Statistic	Mean	Median	Mode	Mean	Median	Mode
Adaptive σ	0.0264	**36.74**	**36.22**	**38.34**	70.89	72.12	77.82
Adaptive σ, best KS	**0.0045**	37.34	36.36	38.88	71.05	71.99	**76.50**
Cross-entropy $\sigma = 0$	0.0293	37.00	36.32	38.74	71.02	72.14	77.65
Cross-entropy $\sigma = 0$, best KS	0.0107	37.44	36.43	39.04	71.15	72.06	77.62
$\sigma = 1$	0.0263	36.83	**36.22**	38.98	70.89	72.01	82.29
$\sigma = 4$	0.0333	36.80	36.26	39.10	70.97	72.12	82.03
$\sigma = 10$	0.0321	36.80	36.23	38.94	**70.88**	**71.92**	80.39
$\sigma = 35$	0.1190	37.06	36.28	43.14	71.43	71.93	93.64

5 Analysis

Discretization exposes the discrete regimes trees capture, allowing attention to concentrate within each bin instead of averaging over them. The quadrature-style adaptive smoothing $\sigma_i = \sqrt{\sigma_{\text{floor}}^2 + (k \cdot w_i)^2}$ keeps the relative smoothing signal consistent by tying it to the target bin width while retaining a floor for narrow bins, which keeps calibration stable even with heterogeneous widths. Treating k and σ_{floor} as tunable hyperparameters lets us optimize the adaptive smoothing alongside the other model-selection variables rather than fixing them separately. This adaptive σ also plays a role similar to the temperature parameter used in post-hoc calibration—it rescales the logits, but unlike a post-training reweighting it is learned during training and therefore remains consistent across the vocabulary and bins. MAE decreases from ≈ 48 s at $h = 1$ to ≈ 30 s at $h = 25$, while attention snapshots show the cadence tokens dominate when entropy is high, confirming they supply the temporal context the model needs. Exploring a sigma-reduction schedule (analogous to a learning-rate decay or simulated annealing) might further help the model navigate flatter minima; we plan to test whether progressively sharpening the Gaussian targets during training accelerates convergence without sacrificing calibration.

6 Discussion and Limitations

Our experiments confirm that respecting discrete regimes lets Transformers rival tuned gradient boosting. The method generalizes to ordinal regression tasks beyond running; any setting with heterogeneous bin widths can adopt the same discretization plus adaptive smoothing recipe. Limitations include the ongoing adaptive sigma experiments (which we are running), the current lack of overflow bins for tail coverage (which can clip predictions when the pace falls outside the discretized vocabulary), and the fact that all results so far derive from the running dataset—external validation (e.g., MIMIC-IV) remains future work. Adding overflow (and underflow) bins that explicitly model $[b_{\max}, \infty)$ and $(-\infty, b_{\min}]$ would let the model gracefully handle previously unseen extreme paces and is part of the planned extensions.

7 Conclusion

Explicit discretization, adaptive Gaussian smoothing, and causal time tokens let a Transformer outperform tuned XGBoost on a large entity-disjoint benchmark while producing better-calibrated PDFs. The paper provides both architectural guidance and analysis tooling (stratified calibration) for future tabular work. One could further treat the adaptive sigma schedule like simulated annealing—starting with larger sigma values and slowly tightening them during training—potentially leveraging the softer targets early on for stability while sharpening distributions as the model converges.

7.1 Learnable Soft Tokenization

Our current approach uses fixed quantile binning for the inputs while applying Gaussian soft targets for outputs. A natural extension would apply the same adaptive Gaussian weighting to the input tokenization, creating a symmetric encoding:

$$\text{embed}(x) = \sum_{i=1}^{K} w_i(x) \cdot \mathbf{e}_i$$

where

$$w_i(x) = \frac{\exp\left(-\frac{(x-c_i)^2}{2\sigma_i^2}\right)}{\sum_j \exp\left(-\frac{(x-c_j)^2}{2\sigma_j^2}\right)}$$

and $\sigma_i = \sqrt{\sigma_{\text{floor}}^2 + (k \cdot \Delta_i)^2}$ uses the same adaptive smoothing formula from Section 3 (where Δ_i is the bin width). This would enable end-to-end learning of optimal bin boundaries while preserving the ordinal structure that discrete regimes provide.

A Additional Analyses

A.1 Runner Trajectories and Calibration Examples

Figure 5 through Figure 7 show three representative runner histories (short, median, and long careers). Each block overlays the observed pace sequence with the Gaussian-smoothed prediction envelopes, illustrating the full predicted distribution (PDF) across the vocabulary and how the density sharpens for consistent performers while broadening for more erratic trajectories.

Runner selection note These three runners were selected not because they are archetypal but because they are known to the authors, which lets us verify that outliers (e.g., YE's slower-than-usual race #7, run with a teenage sibling, or race #19, run with a group of friends) are artifacts of human decision-making rather than modeling failure. The point is to show that even when the model is calibrated, humans still choose to vary their strategy, so distributional forecasts are the right interface for uncertainty-aware guidance.

A.2 Activation and Attention Diagnostics

Activation statistics and attention patterns confirm that cadence tokens dominate when entropy is high, while the pace bins sharpen when variance is low. Figure 8 visualizes the same batch from three views (attention, per-layer contributions, and distributional histogram), showing how the model routes focus toward time deltas during uncertain predictions.

A.3 Data Provenance and Survival-Analysis Context

The core dataset is drawn from the NYRR publicly available results, with the NYRR 9+1 Program ensuring a large, consistent set of races for many athletes [17], and the environmental tokens rely on Visual Crossing's historical weather API [18]. Adaptive sigma caches follow the implementation in the training code, matching the quadrature-style smoothing and $(\sigma_{\text{floor}}, k)$ pairs described earlier.

A.4 Grammar Efficiency Considerations

The current stride grammar repeats entity-level signals such as gender or baseline demographics in every block, which mirrors how the data is stored but adds redundant tokens that could be compressed. A more concise alternative would separate the inherent runner features (fixed across the career) from the event-specific descriptors (distance, weather, pace) and the time tokens, only reintroducing the moving parts at each step. Doing so would preserve the causal story (fixed features $\rightarrow$ event features $\rightarrow$ timing $\rightarrow$ next event features) while saving memory and speeding up training, since fewer repeated embeddings would need to be cached. We keep the redundant copies for transparency and because they simplify batching, but future grammar designs could exploit this structure to cut down on per-stride verbosity.

A.5 Future Work and Applications

Several natural extensions flow from the discretized grammar. One strand treats the pace PDF as a survival-style signal: adding censoring tokens such as `time_censored`, `time_eos`, and `time_missing` can flag trajectories where no future race is observed within the window, so the Transformer can distinguish "still at risk" careers from ones that terminated. This mirrors classic survival analysis [21, 22, 23] while preserving the irregular attention-based representation; mechanisms like right-censoring or overflow bins (e.g., explicit bins for $y < b_1^{start}$ or $y > b_K^{end}$) let the PDF gracefully extend beyond the observed support.

Another strand reinterprets the causal stride as a generative process. A pure generative Transformer can sequentially sample every token within an event block, conditioning on the previous context and respecting hybrid loss functions that treat discrete tokens (categorical weather) with cross-entropy and quantized continuous tokens (pace, numeric environmental features) with smoothed Gaussian penalties. The specific token ordering becomes a hidden hyperparameter in this setup: choosing which environmental covariates to generate first before the outcome changes which conditional factors the model learns, and one could pursue

both conditional generation (treating future covariates as given) and full joint generation (ordering tokens to match a plausible data-generating story).

Such an auto-regressive sampler enables Monte Carlo digital twins: recursively sampling from the predicted PDF at every step produces families of plausible future trajectories, which support stress testing, tail-risk analysis, or scenario planning in domains like energy, flood management, or finance. These trajectories can also be conditioned on arbitrary covariate sequences to simulate missing data (e.g., inserting sentinel tokens for the absence of temperature or opponent identity) or to explore heterogeneous event grammars (race vs training run events). Together, these extensions underscore how the same discretized architecture scales from distributional forecasting to generative simulation while retaining calibration and interpretability.

A.6 Huber-loss exploration

An early experiment combined the Transformer's softmax head with a Huber-style point-loss on the expected pace as an auxiliary objective. The idea was to force the model to hedge around the prediction while still learning a distribution; the result was instructive. Under this composite objective the model learned to route almost all probability mass to the extreme pace bins (first and last of the vocabulary) while adjusting their relative weights to hit whatever point estimate the Huber loss demanded. In other words, the model "cheated" the loss by ignoring the interior bins and collapsing the distribution onto the extremes, so the apparent MAE improved at the cost of losing the predictive PDF entirely. This failure reinforced the lesson that RunTime must prioritize distributional fidelity—hence the switch to full Gaussian-smoothing with cross-entropy as described in the main text.

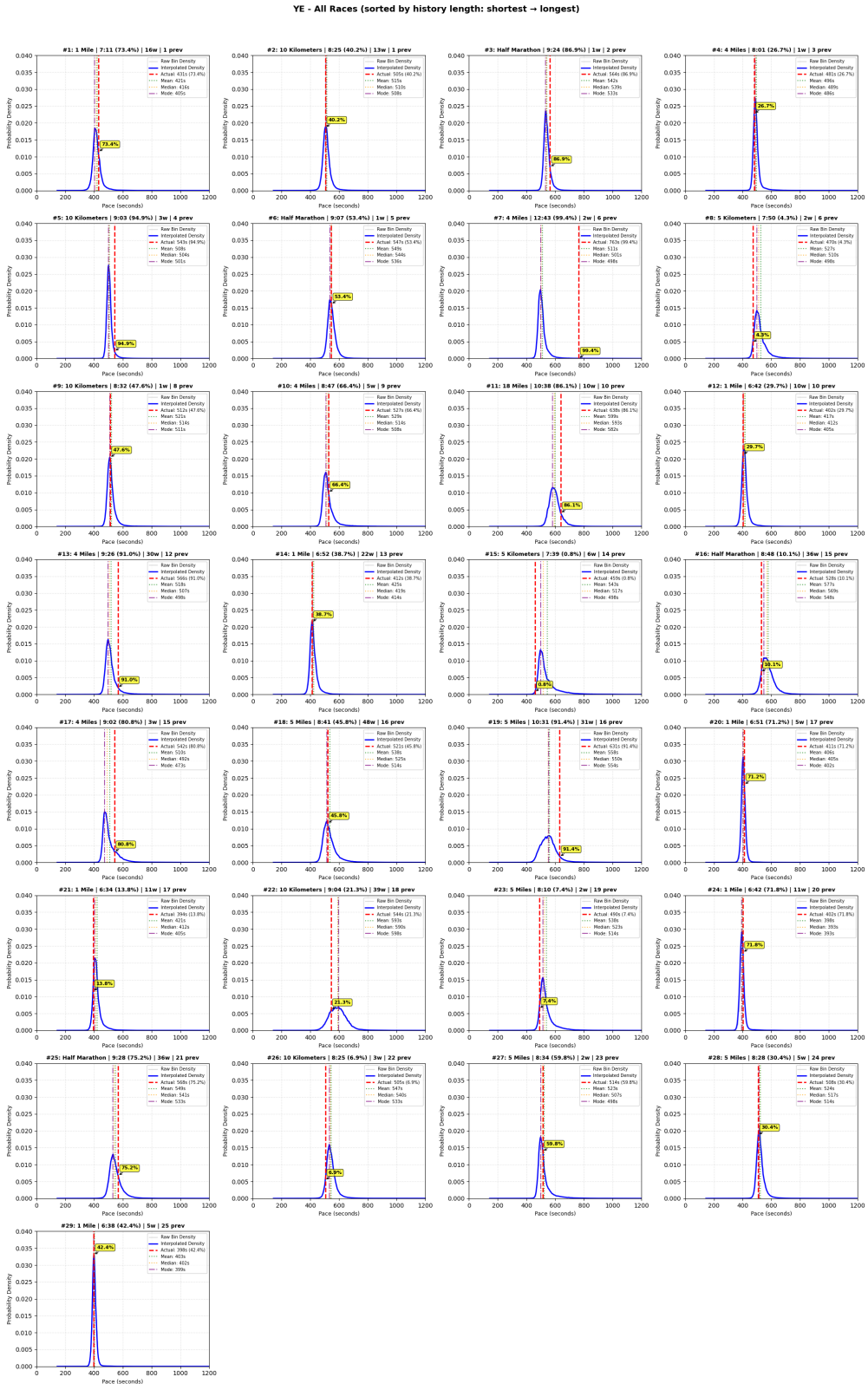

Figure 5: YE (short-career) runner trajectory with predicted PDFs illustrating tight uncertainty as the history grows.

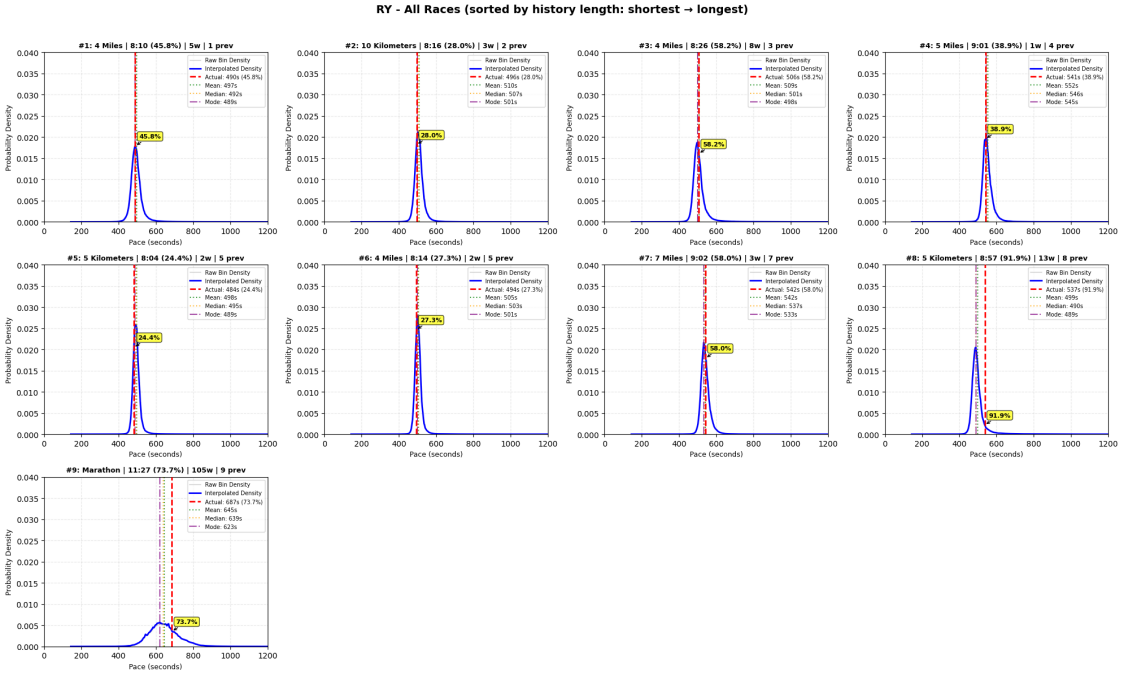

Figure 6: RY (median-career) runner trajectory showing moderate calibration and widening distributions for longer distances.

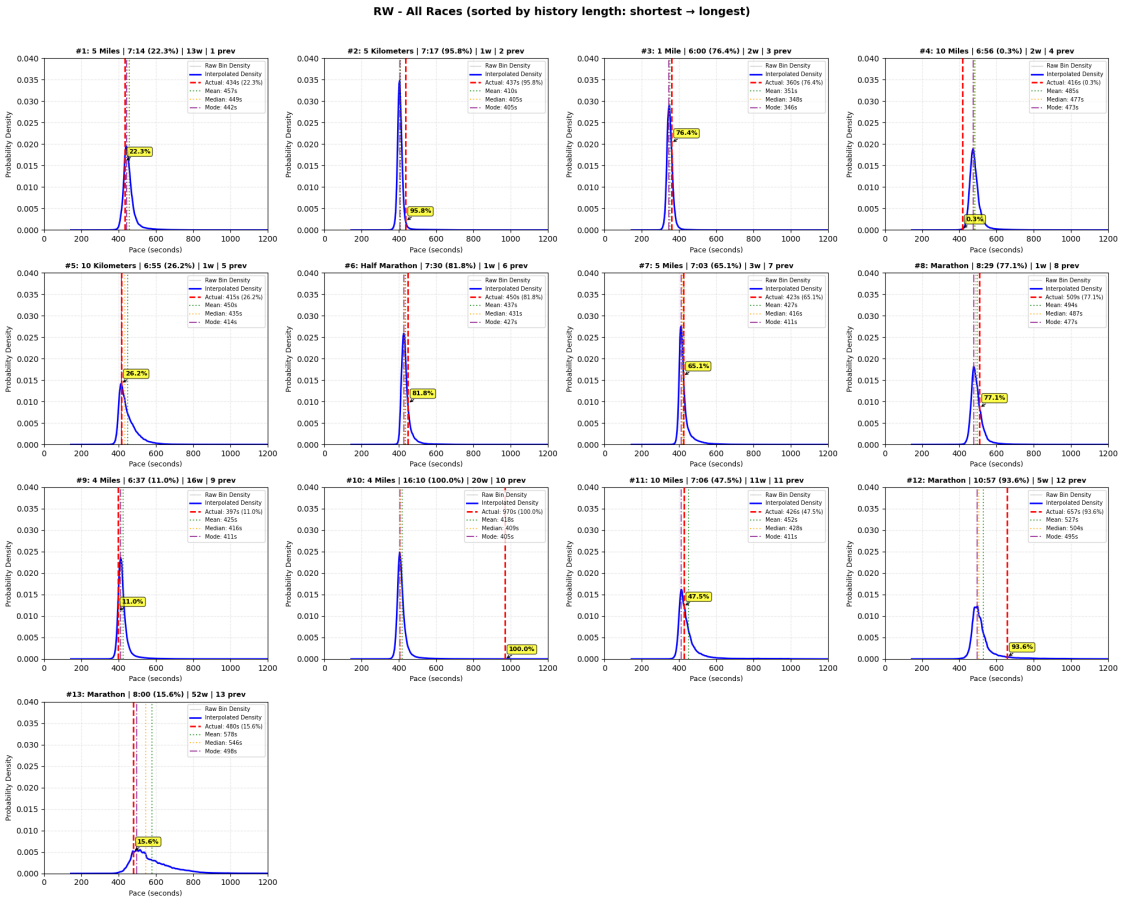

Figure 7: RW (long-career) runner trajectory that exhibits sustained variability and broad PDFs even late in the history.

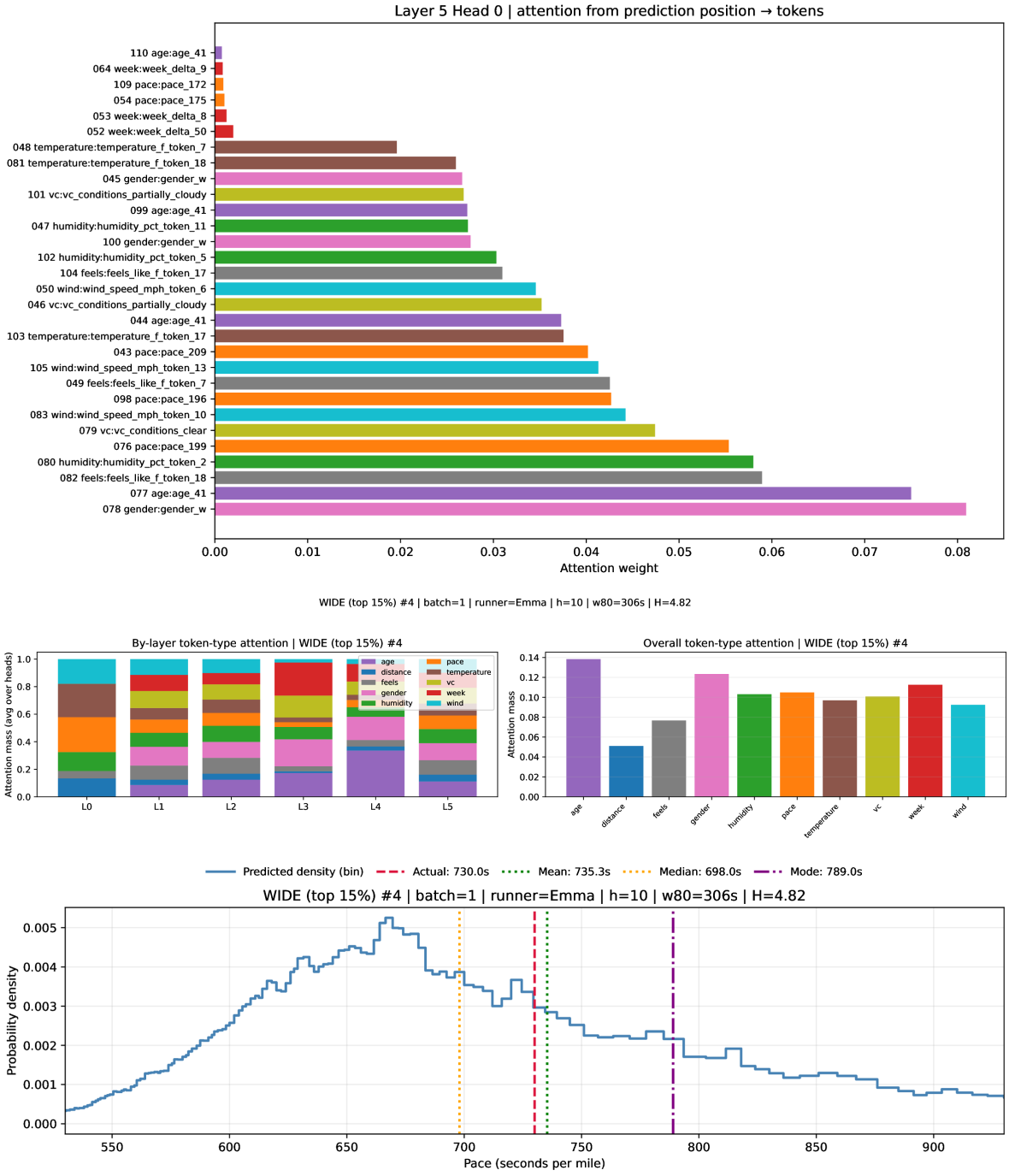

Figure 8: Activation and attention diagnostics for a high-uncertainty prediction. Top: attention weights for the final transformer block. Middle: per-layer contribution magnitudes. Bottom: pace distribution histogram showing the predicted softmax.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[3] Stephan Rabanser, Tim Januschowski, Valentin Flunkert, David Salinas, and Jan Gasthaus. The effectiveness of discretization in forecasting: An empirical study on neural time series models. *NeurIPS*, 2020. arXiv preprint arXiv:2005.10111.

[4] Aamir Fazli Ansari et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.

[5] Aamir Fazli Ansari et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.

[6] Yury Gorishniy, Ivan Rubachev, Viktor Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 2021.

[7] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. Tabtransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678*, 2020.

[8] Sercan Ö. Arik and Tomas Pfister. Tabnet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[9] Leo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS*, 2022.

[10] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 2022.

[11] Raul Diaz and Amit Marathe. Soft labels for ordinal regression. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4733–4742, 2019. doi: 10.1109/CVPR.2019.00487.

[12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the International Conference on Machine Learning*, 2017.

[13] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. *arXiv preprint arXiv:1807.00263*, 2018.

[14] Nan Du et al. Recurrent marked temporal point processes: Embedding event history to vector. In *KDD*, 2016.

[15] Oleksandr Shchur, Marin Biloš, and Stephan Günnemann. Intensity-free learning of temporal point processes. In *ICLR*, 2020.

[16] Simiao Zuo et al. Transformer hawkes process. In *ICML*, 2020.

[17] New York Road Runners. Nyrr 9+1 program. `https://www.nyrr.org/run/guaranteed-entry/tcs-new-york-city-marathon-9-plus-1-program`, 2024.

[18] Visual Crossing. Visual crossing weather data. `https://www.visualcrossing.com/weather-data`, 2024.

[19] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[20] Peter S Riegel. Predicting race times from training levels. *Runner's World*, 17(12):1–5, 1981.

[21] David R Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–202, 1972.

[22] Jared L. Katzman et al. Deepsurv: Personalized treatment recommender system using a cox proportional hazards deep neural network. *BMC Medical Research Methodology*, 18(1):1–12, 2018. doi: 10.1186/s12874-018-0482-1.

[23] Changhee Lee et al. Deephit: A deep learning approach to survival analysis with competing risks. In *AAAI*, 2018.